

When Is a Scale Redundant? Data-Dependent Scale Redundancy and Zero-Training Pruning for Real-Time Object Detection

Yuanchao Yan^{a,*}, Xin Yan^a, Jianlong Liu^a

^a*Department of Artificial Intelligence and Computational Science, Science and Technology College, Gannan Normal University, Ganzhou 341000, China*

Abstract

Real-time detectors run at full capacity on every image, yet where their computation is redundant is rarely quantified. We study YOLO26n on three datasets spanning opposite object-size regimes (VOC, COCO, VisDrone) and report two findings. First, scale redundancy is dataset-dependent: removing the small-object scale P3 costs 0.010/0.032/0.101 mAP on VOC/COCO/VisDrone, whereas the large-object scale P5 costs 0.417/0.176/ ≈ 0 ; per-scale gradient contributions track the small-object fraction, and P3 versus P5 importance reverses between 3.8% and 24.8% small objects (a three-point crossover near 12.6%, not a law). Second, FLOPs $\neq$ FPS: head-level cell skipping yields no GPU speedup, and the $\sim 95\%$ cell redundancy is only exploitable as a mask-level oracle, not as real skipping on GPU. Where a scale is truly redundant, dropping it is a zero-training optimization: on VisDrone, P5 removal cuts FLOPs/parameters by 10.4%/28.6% at identical mAP and reduces single-image latency by 6%. We also report negative results—early exit is infeasible and fine-tuning a dropped scale can hurt—as cautionary evidence for FLOPs-only efficiency claims.

Keywords: real-time object detection, model efficiency, spatial token pruning, early exit, YOLO

1. Introduction

As LLMs that *overthink* spend tokens that do not improve the answer [1, 2], real-time detectors may *oversee*: they compute features the input does not need. Detectors are a clean testbed because their computation decomposes into explicit units at three granularities—grid cells (tokens), feature-pyramid scales, and model checkpoints.

Prior work leaves important gaps: token-pruning methods [3, 4, 5, 6] report large FLOPs savings but target attention models and report FLOPs rather than latency; early-exit detectors [7, 8, 9] attach confidence branches without identifying which features can be omitted; and efficiency studies are almost always single-dataset, leaving the dataset-dependence of redundancy unexamined.

This paper conducts a cross-distribution empirical study with YOLO26n [10] on VOC, COCO, and VisDrone, plus a cross-architecture check with YOLO11n, asking two questions. (RQ1) Is a detection scale redundant, and is the redundant scale stable across datasets? P3 is nearly redundant on VOC but critical on VisDrone, while P5 is completely redundant on VisDrone; gradient shares track the small-object fraction, and the P3/P5 ranking reverses between 3.8% and 24.8% small objects. (RQ2) Can redundancy become real speed? Cell-level redundancy is large (about 95% of grid cells) but, on GPU, only exploitable as a mask-level oracle—real block skipping neither speeds up inference nor preserves accuracy without re-training. Dropping a redundant scale is instead a zero-training, latency-reducing optimization compared against input-resolution scaling and a smaller model.

*Corresponding author

Email addresses: yanyuanchao@gnnustc.com (Yuanchao Yan), yanxin@gnnustc.com (Xin Yan), liujianlong@gnnustc.com (Jianlong Liu)

Contributions: (1) a cross-distribution per-scale contribution analysis showing the redundant scale is data-dependent and tracked by the small-object fraction, with a three-point trend (not a law) reversing the P3/P5 ranking around 12.6%; (2) an honest FLOPs-vs-FPS analysis showing cell-level redundancy is only a mask-level oracle on GPU, plus a zero-training scale-drop recipe compared against input-resolution scaling and a smaller model; (3) negative results (early exit infeasible, fine-tuning a dropped scale can hurt) as cautionary evidence.

2. Related Work

Dynamic token pruning and early exit. DynamicViT [3], EViT [4], ToMe [5], and AdaViT [6] prune or merge tokens adaptively, reporting 30–40% FLOPs savings with negligible accuracy loss on attention models. Self-distillation of importance signals is central to TokenSkip [11] and Self-Calibration [12]. BranchyNet [7], EOBranch [8], and AnytimeYOLO [9] attach early-exit branches to YOLO. These lines of work share our goal—spend computation only where it helps—but report FLOPs rather than latency, target attention models or exits, and do not identify which features a detector can actually omit; for a CNN detector we show the FLOPs claim does not survive head-level cell pruning.

Dynamic computation and pruning in detectors. Dynamic R-CNN [13] and YOLOv7 [14] adapt training targets or label assignment during optimization, Sparse R-CNN [15] and Cascade RPN [16] reduce proposal-level computation, and channel-pruning recipes—Network Slimming [17], AutoSlim [18], and popular YOLOv5 pruning tutorials [19]—remove channels with negligible accuracy loss, including MCCP [20] for multi-collaboration channel pruning. Real-time detectors [21, 22, 23, 10] rely on multi-scale pyramids [24], and cascade [25] or task-aligned [26] designs assume all scales are needed. Two differences distinguish our work. Unlike channel-pruning methods such as Network Slimming [17] or MCCP [20], which find important channels by sparse training or search and then retrain, the proposed scale drop removes an entire feature-pyramid scale with zero training, because the redundant scale is identified from data statistics alone. Unlike dynamic-proposal detectors such as Sparse R-CNN [15], which learn a small set of object proposals, we keep the dense grid and prune the pyramid’s native scales. Finally, we optimize latency (FPS) rather than FLOPs, showing where each type of saving actually transfers on GPU. On the deployment side, the cheapest generic speed-ups—reducing input resolution or removing the largest scale entirely—are common engineering practice; we treat them as first-order baselines in Sec. 4.5.

3. Method

3.1. Contribution analysis

For the FPN-style neck scales P_3, P_4, P_5 [24], the importance of scale i is measured by (i) gradient contribution $G_i = \|F_i \odot \partial L / \partial F_i\|_1$ (normalized), (ii) counterfactual loss change $C_i = (L(Y^{-i}) - L(Y)) / L(Y)$, and (iii) detection-gain ablation $\Delta \text{AP}_i = \text{AP}(F) - \text{AP}(F_{-i})$ using a *fair* head-level mask: the pruned scale produces no detections (scores $\rightarrow -\infty$), avoiding false positives from naive zero-feeding.

3.2. Spatial token budget and self-distillation gate

Following adaptive token pruning [3, 4] and self-distillation of importance signals [11, 12], a gate g_ϕ is a per-scale 1×1 convolution producing cell scores; for budget b we keep the top- b fraction of cells per image and suppress the rest in the head. We compare an *oracle* (rank by the model’s own head scores), an *energy* gate (rank by $\|F_i\|_1$), and a *self-distilled* gate trained with the model frozen via importance-weighted BCE,

$$\mathcal{L}_{\text{gate}} = \sum_i \text{BCE}(g_\phi(P_i), t_i) w_i, \quad t_i = \max_c \sigma(s_{i,c}), \quad w_i = 1 + 30t_i, \quad (1)$$

where $s_{i,c}$ denotes the classification score output by the teacher’s detect head (not the feature-map value) for cell i and class c , and t_i is its max-class confidence. The weight $w_i = 1 + 30t_i$ up-weights high-confidence cells (a cell with $t_i > 0.1$ receives at least $4\times$ the background weight); the gate is trained with Adam (lr $= 10^{-3}$, batch 64) for 5 epochs with the backbone, neck, and head frozen.

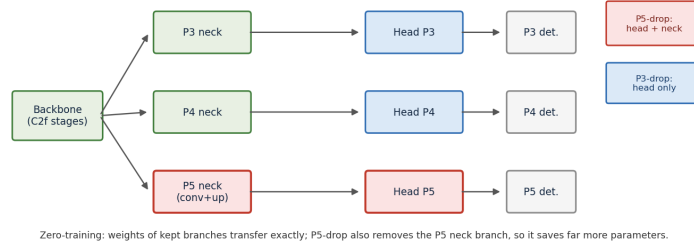


Figure 1: What is dropped. P3-drop removes only the P3 branch of the detect head; P5-drop additionally removes the P5 neck branch (conv + upsampling), which is why it saves far more parameters and FLOPs.

3.3. Zero-training scale drop

Unlike conventional channel pruning, which finds important channels through retraining or search [20, 27], our scale drop needs no optimization: if a scale is redundant ($\Delta\text{AP} \approx 0$), it is removed and the remaining weights are transferred exactly, because removing the redundant tail branch does not change the input channels of the kept head scales. In practice we treat a scale as redundant when removal costs no more than ≈ 1 pp of mAP50-95: P5 on VisDrone costs ≈ 0 (unambiguous), while P3 on VOC costs 1.0 pp (0.010), a borderline case near the evaluation noise measured in Sec. 4.1. Scales costing more than 5 pp (P5 on VOC, 0.417; P3 on VisDrone, 0.101) are clearly necessary. The 1 pp threshold is a rule of thumb to be calibrated to the user’s accuracy budget, not a certified bound.

4. Experiments

4.1. Setup

We use YOLO26n [10] trained with the standard YOLO26n recipe (details in the supplementary material) on VOC [28] (16 551 train/4 952 test2007), COCO [29] (official pretrained weights, val2017), and VisDrone [30] (6 471 train/548 val). Ablations use fair head-level masks with the standard validator; FLOPs are measured on fused models at 640. Latency is single-GPU FP32 on an NVIDIA GeForce RTX 5070 (CUDA 13.2, no TensorRT); we report batch-32 throughput (FPS) and batch-1 per-image latency (ms), since the two can rank optimizations differently.

4.2. RQ1: scale importance is dataset-dependent

Dataset	small obj. (<32px)	G_{P3}	G_{P5}	ΔAP_{P3}	ΔAP_{P5}
VOC	3.8%	22%	39%	0.010	0.417
COCO	24.8%	28%	30%	0.032	0.176
VisDrone	85.9%	53%	7%	0.101	−0.0001

Table 1: Per-scale importance is monotonically tied to the small-object fraction s : gradient contribution G (cols. 3–4) and mAP cost ΔAP of removing a scale (cols. 5–6). The redundant scale flips between VOC (drop P3) and VisDrone (drop P5).

Fitting the three datasets (Fig. 2) gives $G_{P3}(s) = 0.494 + 0.094 \ln s$ and $G_{P5}(s) = 0.096 - 0.098 \ln s$ ($R^2 = 0.81/0.87$), crossing at $s^* \approx 12.6\%$; the mAP cost follows $\Delta\text{AP}_{P3} \approx 0.10 s^{0.73}$. With only three points these are trend descriptions, not laws: per-point residuals are 3.3/8.3/5.0 pp for G_{P3} and 2.6/6.7/4.1 pp for G_{P5} (VOC/COCO/VisDrone), and the crossover lies between the observed points at 3.8% and 24.8%, so 12.6% is an interpolation within our range, not an observed phenomenon. The ranking (P3 least, P5 most important) also holds for YOLO26s/26m on COCO (supplementary material) and for YOLO11n on COCO (ΔAP 0.017/0.052/0.186 for P3/P4/P5), so it is not a nano- or architecture-specific artifact.

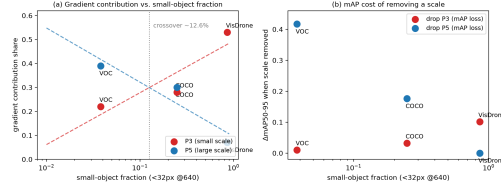


Figure 2: Gradient contribution (a) and mAP cost of scale removal (b) versus small-object fraction; the P3/P5 ranking reverses in our three-point trend.

4.3. RQ2: cell-level redundancy is a mask-level oracle

Cell occupancy (cells containing object centers) is 0.05/0.19/0.74% (VOC) and 1.02/3.52/10.56% (VisDrone) for P3/P4/P5, yet an oracle that keeps the top-5% cells by head score retains essentially all mAP on both datasets (Table 2). This is a mask-level analysis: it suppresses scores without saving compute.

Budget (% cells)	VOC			VisDrone		
	Oracle	Energy	Distilled	Oracle	Energy	Distilled
5%	0.5895	0.171	0.5789	0.1725	0.075	0.1675
10%	0.5896	0.298	0.5852	0.1725	0.109	0.1719
25%	0.5896	0.485	0.5882	0.1725	0.148	0.1724

Table 2: Budget curves (mAP50-95, mask-level): an oracle ranking cells by the model’s own head scores retains essentially all mAP at 5% of cells; the self-distilled gate is within 0.5pp of full accuracy at 10% budget (VOC: 0.5852 vs. 0.5899; VisDrone: 0.1719 vs. 0.1725) and 1.1pp at 5%; the energy gate degrades much more on dense VisDrone. Full 5–75% curves and YOLO26s/26m ablations are in the supplementary material; these numbers assume full features (real skipping: Table 3).

4.4. RQ3: FLOPs $\neq$ FPS on GPU

The detect head is 17.3% of total FLOPs (0.90 of 5.21 G), and preprocessing+postprocessing account for 41% of end-to-end latency at batch 32 (batch-1 model latency 9.4 ms). We implemented real block-skipping of pruned cells (halo-padded gather $\rightarrow$ head convs $\rightarrow$ scatter); batch-32 throughput showed no improvement at any budget (supplementary material):

Config	mAP50-95	GFLOPs
Full	0.5899	5.21
Block-skip 5%	0.381	4.79
Block-skip 25%	0.478	4.94

Table 3: Head-level block skipping neither accelerates inference nor preserves accuracy: halo padding inflates compute and gather/scatter overhead cancels the FLOPs savings; the gap to the mask-level oracle (Table 2) reflects the head seeing pruned features that differ from the full features used to train the gate.

4.5. Zero-training scale drop

We do not pursue difficulty routing or early exit as speed-ups (Sec. 5).

5. Negative results and lessons

Backbone early exit is infeasible on VOC: detection from P3-only or P3+P4 features yields 0.022/0.159 mAP, because deep features are the detection representation itself. Head-level token skipping does not accelerate GPUs (Table 3), a cautionary result for FLOPs-only claims [3, 4]; fine-tuning a redundant-scale drop can hurt (0.169 vs. 0.173).

	mAP50-95	GFLOPs	Params(M)	b1 latency (ms)	b1 FPS
<i>VOC: drop P3 (head-only, zero training)</i>					
Full	0.5899	5.21	2.38	9.42	106
P3-drop	0.5806	4.93	2.36	8.43	119
<i>VisDrone: drop P5 (zero training)</i>					
Full	0.1725	5.20	2.38	10.02	100
P5-drop	0.1725	4.66	1.70	9.40	106

Table 4: Zero-training scale drop keeps mAP identical where the scale is redundant while cutting FLOPs/params and single-image latency; 30-epoch fine-tuning did not improve over transfer-only (0.169 vs. 0.173). P3-drop removes only the P3 head, whereas P5-drop also removes the P5 neck branch (Fig. 1), which is why the savings differ.

	mAP50-95	GFLOPs	b32 FPS	b1 latency (ms)
<i>VOC (small-object fraction 3.8%)</i>				
Full @640	0.5899	5.21	603	9.42
Full @512	0.5759	—	1033	—
P3-drop @640	0.5806	4.93	769	8.43
<i>VisDrone (small-object fraction 85.9%)</i>				
Full @640	0.1725	5.20	634	10.02
Full @512	0.1450	—	910	—
P5-drop @640	0.1725	4.66	670	9.40

Table 5: Scale drop vs. input-resolution scaling. On VisDrone (85.9% objects <32 px), resolution scaling loses 2.8–3.5 pp because it destroys small-object detail, while P5-drop keeps mAP identical and cuts latency by 6%; on VOC, where small objects are rare, resolution scaling is cheaper.

6. Conclusion

Detectors do over-see, but where the redundancy lives is dictated by the data. Across VOC, COCO, and VisDrone, the redundant scale is dataset-dependent—P3 is nearly redundant on VOC and critical on VisDrone, while P5 is completely redundant on VisDrone—and gradient contributions track the small-object fraction, with the P3/P5 ranking reversing between 3.8% and 24.8% small objects (a three-point trend, not a law). Cell-level redundancy is large but, in our measurements, only a mask-level oracle: real block skipping neither speeds up GPU inference nor preserves accuracy without re-training. Where a scale is truly redundant, dropping it is zero-training and cuts single-image latency by 6–12% at identical mAP—the right choice where the cheap generic alternative fails: on dense small-object data (VisDrone), resolution scaling loses 2.8–3.5 pp while P5-drop loses nothing.

Equally important is what does not work: backbone early exit is infeasible, and fine-tuning a redundant-scale drop can hurt. These negative results argue for dataset-aware efficiency and for reporting latency, not just FLOPs. Limitations include three datasets and models up to YOLO26m (plus a YOLO11n architecture check); the $\ln s$ trend and the 12.6% crossover are empirical fits that must be re-validated on remote sensing, medical, or industrial data. Edge-deployment latency (Jetson/TensorRT or mobile SoCs) is explicitly out of scope: all latency numbers are desktop-GPU (RTX 5070), and measuring the zero-training recipe on real edge hardware is left to future work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Yuanchao Yan: Conceptualization, Methodology, Software, Validation, Writing – original draft. **Xin Yan:** Investigation, Data curation, Validation. **Jianlong Liu:** Writing – review & editing, Supervision.

Data availability

All datasets used in this study (PASCAL VOC, MS COCO, and VisDrone) are publicly available from their official sources. Code and models will be made available from the corresponding author on reasonable request.

References

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [2] S. Feng, G. Fang, X. Ma, X. Wang, Efficient reasoning models: A survey, *Transactions on Machine Learning Research* (2025).
- [3] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, C.-J. Hsieh, DynamicViT: Efficient vision transformers with dynamic token sparsification, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [4] Y. Liang, C. Ge, Z. Tong, Y. Song, J. Wang, P. Xie, Evit: Expediting vision transformers via token reuse, in: *International Conference on Learning Representations (ICLR)*, 2022.
- [5] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, J. Hoffman, Token merging: Your ViT but faster, in: *International Conference on Learning Representations (ICLR)*, 2023.
- [6] L. Meng, H. Li, B.-C. Chen, S. Lan, Z. Wu, Y.-G. Jiang, S.-N. Lim, AdaViT: Adaptive vision transformers for efficient image recognition, in: *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [7] S. Teerapittayanon, B. McDanel, H.-T. Kung, BranchyNet: Fast inference via early exiting from deep neural networks, in: *International Conference on Pattern Recognition (ICPR)*, 2016.
- [8] D. Scholte, M. H. Zwemer, E. Bondarev, Accelerating YOLO with EOBranch: An early exit approach for adaptive object detection, in: *International Conference on Image Analysis and Processing (ICIAP)*, 2025.
- [9] R. Menezes, E. Filgueiras, J. Silveira, et al., AnytimeYOLO: Early exits for interruptible object detection, *arXiv preprint arXiv:2503.17497* (2025).
- [10] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, M. E. Kalfaoglu, Ultralytics yolo26: Unified real-time end-to-end vision models (2026). [arXiv:2606.03748](https://arxiv.org/abs/2606.03748), [doi:10.48550/arXiv.2606.03748](https://doi.org/10.48550/arXiv.2606.03748).
- [11] H. Xia, Y. Li, C. T. Leong, W. Wang, W. Li, TokenSkip: Controllable chain-of-thought compression in llms, *arXiv preprint arXiv:2502.12067* (2025).
- [12] Y. Huang, et al., Self-calibration for efficient reasoning, *arXiv preprint arXiv:2503.00210* (2025).
- [13] H. Zhang, H. Chang, B. Ma, N. Wang, X. Chen, Dynamic R-CNN: Towards high quality object detection via dynamic training, in: *European Conference on Computer Vision (ECCV)*, Springer, 2020, pp. 260–275.
- [14] C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.
- [15] P. Sun, R. Zhang, Y. Jiang, T. Kong, C. Xu, W. Zhan, M. Tomizuka, L. Li, Z. Yuan, C. Wang, et al., Sparse R-CNN: End-to-end object detection with learnable proposals, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14454–14463.
- [16] T. Vu, H. Jang, T. X. Pham, C. D. Yoo, Cascade RPN: Delving into high-quality region proposal networks with adaptive convolution, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, C. Zhang, Learning efficient convolutional networks through network slimming, in: *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2736–2744.
- [18] J. Yu, T. Huang, AutoSlim: Towards one-shot architecture search for channel numbers, in: *International Conference on Machine Learning (ICML)*, 2020, pp. 10862–10872.
- [19] G. Jocher, et al., Ultralytics YOLOv5, <https://github.com/ultralytics/yolov5> (2023).
- [20] Y. Yan, B. Liu, W. Lin, Y. Chen, K. Li, J. Ou, C. Fan, MCCP: Multi-collaboration channel pruning for model compression, *Neural Processing Letters* 55 (3) (2023) 2777–2797.
- [21] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [22] Z. Tian, C. Shen, H. Chen, T. He, FCOS: Fully convolutional one-stage object detection, in: *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [23] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2017.
- [24] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [25] Z. Cai, N. Vasconcelos, Cascade R-CNN: Delving into high quality object detection, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [26] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, W. Huang, Tood: Task-aligned one-stage object detection, in: *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [27] S. Han, J. Pool, J. Tran, W. Dally, Learning both weights and connections for efficient neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [28] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman, The Pascal Visual Object Classes (VOC) challenge, *International Journal of Computer Vision* 88 (2) (2010) 303–338.
- [29] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft COCO: Common objects in context, in: *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 740–755.
- [30] P. Zhu, L. Wen, D. Du, X. Bian, H. Ling, Q. Hu, J. Nie, H. Cheng, C. Liu, X. Ma, et al., Detection and tracking meet drones challenge, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (11) (2022) 7380–7399.